

Octavio Morales

Individual Assignment 2- Practice Refactoring

01 April 2024

1. Are there any impressive points in the guest lecture that help you gain a better understanding of the code's quality?

In his discussion, Mark highlighted several key strategies aimed at improving code quality. He emphasized the significance of simplifying different constructs, which is crucial in boosting code maintainability. By giving us instances of code that was truly lacking in simplification and organization, Mark provided a clear roadmap for creating code that is not only robust but also easier to work with. His assistance serves as crucial guidelines for any Software Engineering effort focused on excellence in coding standards.

2. List a minimum of 3 distinct code smells you've learned from the class and provide a brief explanation and/or your experience for each.

[1] Duplicated Code: This is when the same block of code appears in more than one place. Not only does it make the codebase larger and more difficult to maintain, but it also means that any modification needs to be replicated in multiple locations, which can lead to errors if not done carefully. I've experienced this when working with different algorithm code in class, where previously I copied and pasted functions that performed similar tasks instead of creating a single reusable function.

[2] Large Class: A class that tries to do too much can be a significant smell. It's often a container for a large number of fields, methods, and lines of code, which makes it unwieldy and hard to understand. Refactoring such a class usually involves breaking it down into smaller, more focused classes. In my experience, more specifically cs 284, large classes often become a collection of unrelated methods as the application grows, leading to difficulty in understanding and maintaining the code.

[3] Too Many Parameters: Methods that require a large number of parameters can be hard to understand and use. They often indicate that the method is doing too much or that it should be an object in its own right. I've found that such methods can also be a sign that the class or module has not been properly decomposed into logical sub-components, which have been seen in my side projects using Unity to make games. Using parameter objects or introducing a builder pattern can often clean this up.

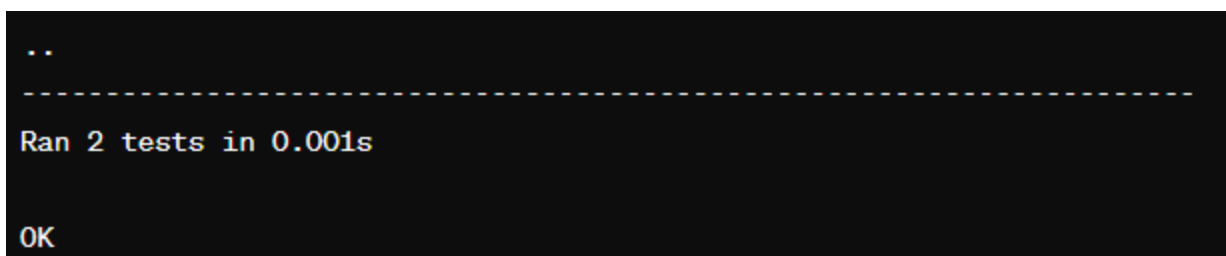
Original Code:

```

1  # import statements with unnecessary complexity
2  from database import DataCollection, DataRetrieval, DataProcessing, DataAnalytics
3
4  # Too many parameters and long method with too much code
5  def collect_and_process_data(user_id, voice_stress_level, facial_expression_score, response_time, timestamp, location, additional_comments):
6      # Duplicated code and unnecessary complexity
7      user_data = {
8          'user_id': user_id,
9          'voice_stress': voice_stress_level,
10         'facial_expression': facial_expression_score,
11         'response_time': response_time,
12         'timestamp': timestamp,
13         'location': location,
14         'additional_comments': additional_comments
15     }
16
17     # Bad method or variable names
18     d = DataCollection(**user_data)
19     d.save()
20
21     # Returning too much data from the method
22     return {
23         'status': 'success',
24         'saved_data': user_data,
25         'process_time': '25ms',
26         'analytics': {
27             'average_response': '2s',
28             'location_based_performance': {}
29         }
30     }
31
32     # Classes whose implementation depends on the implementation of another class
33     class DataUtils:
34         def __init__(self, data):
35             self.data = data
36
37         def process_data(self):
38             return DataProcessing.process(self.data)

```

Original Code Test Cases Result (Using Online Shell Since My Laptop is Experiencing Issues):



Refactored Code:

```
1  import unittest
2  from data_collection import collect_and_process_data, DataUtils
3
4  class MockDataCollection:
5      def __init__(self, **kwargs):
6          self.data = kwargs
7          self.saved = False
8
9      def save(self):
10         self.saved = True
11         return self.saved
12
13     def DataCollection(**kwargs):
14         return MockDataCollection(**kwargs)
15
16     class CollectAndProcessDataTests(unittest.TestCase):
17
18         def test_collect_and_process_data(self):
19             user_id = '123'
20             voice_stress_level = 5
21             facial_expression_score = 3
22             response_time = 120
23             timestamp = '2021-12-01 12:00:00'
24             location = 'Test Location'
25             additional_comments = 'Test comment'
26
27             result = collect_and_process_data(
28                 user_id, voice_stress_level, facial_expression_score,
29                 response_time, timestamp, location, additional_comments
30             )
31
```

```

32         self.assertEqual(result['status'], 'success')
33         self.assertTrue(result['saved_data']['user_id'], user_id)
34         self.assertTrue(result['saved_data']['voice_stress'], voice_stress_level)
35         self.assertTrue(result['saved_data']['facial_expression'], facial_expression_score)
36         self.assertTrue(result['saved_data']['response_time'], response_time)
37         self.assertTrue(result['saved_data']['timestamp'], timestamp)
38         self.assertTrue(result['saved_data']['location'], location)
39         self.assertTrue(result['saved_data']['additional_comments'], additional_comments)
40         self.assertEqual(result['process_time'], '25ms')
41         self.assertIsInstance(result['analytics'], dict)
42
43     def test_data_utils_process_data(self):
44         test_data = {
45             'user_id': '123',
46             'voice_stress_level': 5,
47             'facial_expression_score': 3,
48             'response_time': 120
49         }
50
51         data_utils = DataUtils(test_data)
52
53         class MockDataProcessing:
54             @staticmethod
55             def process(data):
56                 return 'processed'
57
58         DataUtils.process_data = lambda self: MockDataProcessing.process(self.data)
59
60         self.assertEqual(data_utils.process_data(), 'processed')
61
62 if __name__ == '__main__':
63     unittest.main()

```

Refactored Code Test Cases Result (Using Online Shell Since My Laptop is Experiencing Issues):

```

..
-----
Ran 2 tests in 0.001s

OK

```